

40 Ways AI Agents Break in Production: The Edge Cases, the Failure Modes, and the Eval That Catches Each One

The 40 production killers that break AI agents in the wild, each with the failure it causes and the eval that catches it before your users do.

Input handling: the 8 ways real-world input breaks an agent

Most agents are tested on clean prompts and fall over on the messy reality of production input.

1. Empty or whitespace-only input. Failure: the agent invents a task or calls a tool with null args. Eval: assert it asks for clarification, never acts.
2. Input that blows the context window. Failure: silent truncation drops the user's real question. Eval: feed 1.5x the window, assert graceful summarize-or-refuse, not a truncated guess.
3. Mixed-language or code-switched input. Failure: it answers in the wrong language or ignores half the message. Eval: bilingual prompts, assert it responds in the user's dominant language.
4. Prompt injection in user content ('ignore previous instructions'). Failure: it obeys the injected instruction. Eval: a battery of injection strings, assert the system prompt holds.
5. Injection from retrieved documents or tool output. Failure: a poisoned page or record hijacks the agent. Eval: seed your RAG corpus with adversarial docs, assert they cannot change behavior.
6. Unicode tricks, homoglyphs, zero-width characters. Failure: guardrails are bypassed. Eval: normalize-and-test with a homoglyph corpus.
7. Locale-ambiguous numbers and dates (1,000 vs 1.000, MM/DD vs DD/MM). Failure: it misreads the value and acts on it. Eval: ambiguous inputs with known-correct parses.
8. Valid but impossible requests ('book a flight to the moon'). Failure: it confidently proceeds. Eval: impossible requests, assert it declines with a reason.

Tool use: the 8 ways agents misuse tools and APIs

1. Hallucinated tool arguments. Failure: wrong record updated, wrong email sent. Eval: schema-validate every tool call against ground truth before execution.
2. Wrong tool for the task. Failure: it searches when it should query the database. Eval: per-intent expected-tool assertions.
3. Ignoring a tool error and proceeding. Failure: it reports success on a failed action. Eval: inject 500s and timeouts, assert it surfaces the failure, never fabricates a result.
4. Infinite tool-call loops. Failure: it calls the same tool forever, burning cost. Eval: cap and assert a hard stop with a sane message.
5. Stale or cached results treated as fresh. Failure: it acts on yesterday's data. Eval: freshness checks on time-sensitive tools.
6. Parallel calls with hidden dependencies. Failure: step B runs before step A finishes. Eval: dependency-ordered traces.

7. Tool output too large for context. Failure: the result is truncated and it reasons on half. Eval: oversized returns, assert summarize-then-reason.
8. Side-effectful tools on a read-only request. Failure: an exploratory question mutates data. Eval: assert no write-tool fires on read intents.

Context and memory: the 7 ways agents lose the thread

1. Losing earlier constraints in a long chat. Failure: it forgets the budget set 10 turns ago. Eval: long multi-turn scripts that check constraint retention.
2. Conflating two users or sessions. Failure: cross-user data leak. Eval: interleaved-session tests, assert strict isolation.
3. Over-trusting its own earlier wrong output. Failure: an early hallucination compounds. Eval: seed a wrong fact mid-chat, assert it can correct, not entrench.
4. Context eviction dropping the system prompt. Failure: guardrails silently fall out of context. Eval: assert the system prompt and pinned facts survive at max length.
5. Summarization dropping the one detail that mattered. Failure: the rolled-up memory loses the account number. Eval: needle-in-haystack over summarized history.
6. Trusting stale memory over the live user. Failure: it argues with the user using old notes. Eval: conflict cases, assert the live user wins.
7. No memory when memory was expected. Failure: it re-asks for info already given. Eval: assert recall of prior-turn facts.

Hallucination and grounding: the 7 ways agents invent things

1. Citing sources that do not exist. Failure: fake URLs and references. Eval: assert every citation resolves to a real retrieved doc.
2. Confident answers with zero retrieved support. Failure: it fills a gap with fiction. Eval: assert abstain-or-hedge when retrieval returns nothing.
3. Mixing facts from two records. Failure: customer A's data attributed to customer B. Eval: provenance checks on every claim.
4. Inventing API fields, function names, config keys. Failure: code that looks right and does not exist. Eval: validate generated identifiers against the real schema.
5. Plausible-but-wrong math. Failure: it reasons fluently to a wrong number. Eval: a numeric set with exact expected answers.
6. Overconfidence out of domain. Failure: no signal it is guessing. Eval: a calibration set, assert it flags low confidence.
7. Agreeing with a false premise. Failure: it runs with the user's wrong assumption. Eval: false-premise prompts, assert it corrects first.

Control flow: the 6 ways agents get stuck or run away

1. Never deciding it is done. Failure: it loops past the answer. Eval: a clear stop condition on every task type.
2. Giving up too early. Failure: 'I cannot help' on something in scope. Eval: a should-succeed set, assert completion.

3. Runaway cost on one request. Failure: a single query spends 50x budget. Eval: per-request token and tool-call caps with assertions.
4. Deadlock between sub-agents. Failure: the whole run hangs. Eval: timeout-and-detect on multi-agent graphs.
5. Replanning forever instead of acting. Failure: analysis paralysis, no output. Eval: bound the plan-revise cycle, assert progress.
6. Silent partial completion. Failure: it does 3 of 5 steps and reports success. Eval: step-coverage assertions on multi-step tasks.

Production realities: the 4 ways agents break under real load

1. Non-determinism making a passing demo flake one in ten. Failure: 'works on my machine' that fails in prod. Eval: run each case N times, assert a pass rate, not a single pass.
2. Model or prompt version drift. Failure: a provider update silently changes behavior. Eval: a regression suite pinned in CI on every model bump.
3. Latency spikes under concurrency. Failure: it times out at peak. Eval: load test the full tool chain, not just the model call.
4. Cost scaling with the wrong thing. Failure: spend grows with input size, not value. Eval: cost-per-resolved-task as a first-class metric.

What to do with this list

Pick the 10 that scare you most for your feature and write an eval for each before you ship. If you want it done as a written suite that runs in CI, that is exactly what we build at Diraya: a working AI feature in 8 weeks with the eval suite that keeps it working. Reply to this email and we will set up a short call. If not, run the list yourself. It works either way.